

Two-Phase Hybrid Algorithm for the Dutch Merchant Problem

Abstract

This document presents the design, implementation, and analysis of a two-phase hybrid algorithm for solving the Dutch Merchant Problem with instances of size $15 \leq n \leq 30$ ports, supporting multiple item types per port (m items) and operations involving k units per transaction. The algorithm separates route generation (using Prize-Collecting TSP-inspired heuristics) from transaction optimization (using multi-dimensional dynamic programming), achieving efficient solutions for medium-to-large instances while maintaining theoretical foundations in both PCTSP and DP literature. This work corresponds to the efficient algorithm implementation phase of the project, designed to handle larger instances than exact brute force methods can't solve.

1 Introduction

This document presents an efficient algorithm for solving the Dutch Merchant Problem, designed to handle instances with $15 \leq n \leq 30$ ports, where n represents the total number of ports including Amsterdam. The problem extends the basic formulation to support multiple item types per port (m items) and operations that allow buying or selling k units per transaction, significantly increasing the complexity compared to the single-item, single-unit case.

The Dutch Merchant Problem, as formalized in the reduction analysis (Reducción.pdf), is an NP-Complete optimization problem that combines routing decisions with transaction optimization under multiple constraints: capacity limits, capital requirements, and time restrictions. The problem's relationship to the Prize-Collecting Traveling Salesman Problem (PCTSP) [1] provides a natural framework for route generation, while the transaction optimization subproblem exhibits optimal substructure suitable for dynamic programming techniques.

2 Problem Formalization

2.1 Extended Problem Definition

An instance of the extended Dutch Merchant Problem is defined by the tuple $(G, r, \mathcal{M}, \mathcal{P}, B, T_{max}, f, k)$ where:

- $G = (V, E)$ is a complete undirected graph representing the port network. $V = \{v_0, v_1, \dots, v_n\}$, with v_0 being the origin port (Amsterdam).
- $r \in \mathbb{R}^+$ is the initial capital.
- $\mathcal{M} = \{m_1, m_2, \dots, m_{|\mathcal{M}|}\}$ is the set of $m = |\mathcal{M}|$ item types available at ports.
- $\mathcal{P} = \{(c_{v,m}^{compra}, c_{v,m}^{venta}) \mid v \in V \setminus \{v_0\}, m \in \mathcal{M}\}$ defines the purchase and sale prices for each item type at each port. In our implementation, this is represented as dictionaries: `purchase_prices[v][m]` and `sale_prices[v][m]`.
- $B \in \mathbb{N}$ is the maximum cargo capacity of the ship (total units across all item types).
- $T_{max} \in \mathbb{R}^+$ is the maximum total time allowed for the expedition.
- $f : E \rightarrow \mathbb{R}^+ \times \mathbb{R}^+$ assigns to each edge $e = (u, v)$ a travel time t_{uv} and an operational cost c_{uv} .
- $k \in \mathbb{N}$ is the maximum number of units that can be bought or sold in a single operation at a port.

2.2 Decision Space

At each port $v \in V \setminus \{v_0\}$, for each item type $m \in \mathcal{M}$, the captain can choose:

- **Buy operation:** Purchase j units where $j \in \{0, 1, 2, \dots, k\}$ (where $j = 0$ represents doing nothing for this item)
- **Sell operation:** Sell j units where $j \in \{1, 2, \dots, k\}$ (if sufficient inventory exists)

The decision space per port is therefore $O((k+1)^m)$ when considering all item types simultaneously, though in practice we process items sequentially to manage complexity.

2.3 Solution Feasibility

A solution consists of:

1. A cycle $(v_0, v_{i_1}, v_{i_2}, \dots, v_{i_l}, v_0)$ visiting a subset of ports.
2. For each visited port v_{i_j} , a sequence of decisions specifying buy/sell operations for each item type.

A solution is **feasible** if:

1. **Capacity constraint:** At no point does the total cargo load exceed B : $\sum_{m \in \mathcal{M}} \text{load}_m \leq B$ at all times.
2. **Capital constraint:** After each operation and travel, capital remains non-negative: $c_{\text{current}} \geq 0$.
3. **Time constraint:** Total time (travel + operations) does not exceed T_{max} : $\sum_{e \in \text{tour}} t_e + \sum_{\text{ports}} \text{operation_time} \leq T_{\text{max}}$.
4. **Inventory constraint:** Cannot sell more units than currently carried: for each item m , $\text{sell_quantity}_m \leq \text{current_load}_m$.

2.4 Objective

The objective is to maximize the final capital upon returning to Amsterdam (v_0):

$$\max \left\{ r + \sum_{\text{sales}} c_{v,m}^{\text{venta}} \cdot \text{units_sold} - \sum_{\text{purchases}} c_{v,m}^{\text{compra}} \cdot \text{units_bought} - \sum_{e \in \text{tour}} c_e \right\}$$

3 Algorithm Selection and Justification

3.1 Complexity Analysis

The problem exhibits two primary sources of complexity:

1. **Route Selection:** The number of possible routes is $O(n!)$ when considering all permutations of port subsets. This is NP-hard, as established by the reduction from Prize-Collecting TSP in Reducción.pdf (Section 2.3).

2. **Transaction Optimization:** For a fixed route of length L visiting L intermediate ports, with m item types and operations involving up to k units, the decision space is $O((k+1)^{m \cdot L})$. Without structure exploitation, this grows exponentially.

A naive approach combining both would result in complexity $O(n! \times (k+1)^{m \cdot n})$, which is intractable even for moderate values of n , m , and k .

3.2 Why Two-Phase Hybrid Approach?

The two-phase hybrid approach separates these two sources of complexity, applying the most appropriate technique to each:

1. **Phase 1 - Route Generation:** Uses heuristic methods inspired by Prize-Collecting TSP algorithms to generate promising routes without exhaustive enumeration. This leverages the PCTSP structure where ports represent nodes with associated prizes (potential profits) and edges have costs (travel expenses).
2. **Phase 2 - Transaction Optimization:** For each generated route, uses exact dynamic programming to solve the transaction optimization problem optimally. This exploits the optimal substructure: the best transaction decisions at port i depend only on the current state (capital, load vector) and future ports, not on past decisions.

3.3 Connection to Prize-Collecting TSP

The reduction analysis (Reducción.pdf, Section 2.3) establishes that the Dutch Merchant Problem is NP-hard via reduction from Prize-Collecting TSP. This connection provides a natural framework for route generation:

- **PCTSP Structure:** In PCTSP, we seek a cycle starting at root s that visits a subset of nodes to maximize $\sum_{u \in S} \pi_u - \sum_{e \in \text{tour}} \omega_e$, where π_u is the prize for visiting node u and ω_e is the edge cost.
- **Adaptation to Dutch Merchant:** Each port v has an associated potential profit (prize) that can be estimated as:

$$\pi_v = \max_{\text{feasible transactions}} \left\{ \sum_m (c_{v,m}^{\text{venta}} - c_{v,m}^{\text{compra}}) \cdot \text{optimal_units}_m \right\}$$

- **Bound Calculation:** For a route R , we can compute an upper bound using PCTSP-style reasoning:

$$\text{bound}(R) = \sum_{v \in R} \pi_v - \sum_{e \in R} c_e$$

This bound helps prune routes that cannot possibly yield optimal solutions, even with perfect transaction optimization.

3.4 Dynamic Programming Foundation

The transaction optimization subproblem, when the route is fixed, exhibits optimal substructure suitable for dynamic programming. This follows from standard DP principles (Conferencia 3 - Programación Dinámica.pdf, I2A.pdf):

- **Optimal Substructure:** If we know the optimal transaction decisions for ports $i + 1, i + 2, \dots, L$ given any state $(\text{capital}, \text{load_vector})$, then the optimal decision at port i can be determined by considering all feasible actions and selecting the one that leads to the best outcome.
- **State Space:** The state is defined as $(\text{port_index}, \text{capital_level}, \text{load_vector})$ where:
 - $\text{port_index} \in \{0, 1, \dots, L\}$: Current position in the route
 - $\text{capital_level} \in \mathbb{R}^+$: Current capital (may be discretized for efficiency)
 - $\text{load_vector} \in [0, B]^m$: Current cargo load for each item type
- **Value Function:** $V(i, c, \vec{\ell})$ represents the maximum final capital achievable starting from port i with capital c and load vector $\vec{\ell}$.
- **Recurrence Relation:**

$$V(i, c, \vec{\ell}) = \max_{\text{feasible actions } a} \left\{ V(i + 1, c', \vec{\ell}') \right\}$$

where c' and $\vec{\ell}'$ are the capital and load after applying action a at port i and traveling to the next port.

3.5 Advantages of Separation

Separating route generation from transaction optimization provides several advantages:

1. **Controlled Exploration:** Route generation can use heuristic methods (beam search, branch-and-bound) to explore the most promising routes without exhaustive enumeration, limiting exploration to $O(K)$ routes where $K \ll n!$.
2. **Exact Transaction Optimization:** For each route, we solve the transaction problem exactly using DP, ensuring that given a route, we find the optimal transaction decisions. This guarantees that if the route generation includes the optimal route, the overall solution will be optimal.
3. **Flexibility:** The approach allows independent optimization of each phase:
 - Route generation can use various heuristics (greedy, beam search, branch-and-bound with PCTSP bounds)
 - Transaction DP can use different state representations (exact, discretized, sparse) depending on instance characteristics
4. **Scalability:** The complexity becomes $O(K \times L \times m \times k \times B^m \times K_{capital})$ where K is the number of routes explored (controlled), L is route length, and $K_{capital}$ is the number of capital levels (may be discretized). This is polynomial in the route generation parameter K , unlike the exponential $n!$ in brute force.

4 Algorithm Design and Pseudocode

4.1 Algorithm Architecture

The two-phase hybrid algorithm consists of two main components that operate sequentially:

1. **Phase 1:** Generate promising routes using PCTSP-inspired heuristics
2. **Phase 2:** For each route, solve the transaction optimization problem using multi-dimensional dynamic programming

4.2 Phase 1: Route Generation with Beam Search

4.2.1 Overview

Phase 1 uses beam search with PCTSP-style bounds to generate a limited set of promising routes. Beam search maintains a beam (set) of the best partial routes at each depth, expanding only the most promising candidates.

4.2.2 PCTSP Bound Calculation

For a partial or complete route $R = [v_0, v_{i_1}, \dots, v_{i_l}, v_0]$, we calculate an upper bound on achievable profit:

Function CalculatePCTSPBound(route R , instance):

1. total_prize $\leftarrow 0$, total_cost $\leftarrow 0$
2. **For each** port v in R (excluding v_0):
 - $\pi_v \leftarrow \max_{m \in \mathcal{M}} \{ (c_{v,m}^{venta} - c_{v,m}^{compra}) \cdot \min(k, B) \}$ (Maximum profit per port)
 - total_prize $\leftarrow$ total_prize + π_v
3. **For each** edge (u, v) in route:
 - total_cost $\leftarrow$ total_cost + c_{uv}
4. **Return** total_prize – total_cost

4.2.3 Beam Search Algorithm

Function GeneratePromisingRoutes(instance, beam_width K , max_depth):

1. Initialize beam $\leftarrow \{(\text{route} = [0], \text{bound} = 0, \text{time} = 0, \text{visited} = \{0\})\}$
2. best_known $\leftarrow -\infty$, final_routes $\leftarrow \emptyset$
3. **For** depth = 1 to max_depth:
 - (a) candidates $\leftarrow \emptyset$
 - (b) **For each** state s in beam:
 - i. **For each** unvisited port $v \in V \setminus s.\text{visited}$:
 - A. new_route $\leftarrow s.\text{route} + [v]$, new_time $\leftarrow s.\text{time} + t_{\text{last}(s.\text{route}), v}$
 - B. **If** new_time $\leq T_{max}$ and CalculatePCTSPBound(new_route, instance) $>$ best_known – threshold:
 - candidates.append((new_route, bound, new_time, $s.\text{visited} \cup \{v\}$))
 - ii. **If** $|s.\text{route}| > 1$: add complete route to final_routes if time feasible
 - (c) beam $\leftarrow$ top K candidates by bound, update best_known
4. **Return** final_routes sorted by bound (descending)

4.3 Phase 2: Multi-Dimensional Transaction DP

4.3.1 State Representation

For a fixed route $R = [v_0, v_{i_1}, \dots, v_{i_L}, v_0]$ of length $L + 2$ (including start and end at Amsterdam), we define:

- **State:** $(i, c, \vec{\ell})$ where:
 - $i \in \{0, 1, \dots, L\}$: Port index in route ($0 = \text{Amsterdam}$, $i \geq 1 = \text{intermediate ports}$)
 - $c \in \mathbb{R}^+$: Capital level (may be discretized into bands for efficiency)
 - $\vec{\ell} = (\ell_1, \ell_2, \dots, \ell_m) \in [0, B]^m$: Load vector, where ℓ_j is units of item type j carried
- **Value Function:** $V(i, c, \vec{\ell}) = \text{maximum final capital achievable from state } (i, c, \vec{\ell})$

4.3.2 DP Recurrence

The recurrence relation processes ports sequentially:

Function SolveTransactionsDP(route R , instance):

1. $L \leftarrow |R| - 2$, $m \leftarrow |\mathcal{M}|$, $B \leftarrow \text{capacity}$, $k \leftarrow \text{max_units_per_op}$
2. Initialize $dp[0][r][(0, \dots, 0)] \leftarrow r$ (Start at Amsterdam)
3. Travel to first port: $dp[0][r - \text{costs}[0][R[1]]][(0, \dots, 0)] \leftarrow r - \text{costs}[0][R[1]]$ (if ≥ 0)
4. **For** $i = 0$ to $L - 1$:
 - (a) $port \leftarrow R[i + 1]$, $dp_next \leftarrow \emptyset$
 - (b) **For each** state $(c, \vec{\ell})$ in $dp[i]$: generate all feasible actions (do nothing, buy, sell) for all items, update dp_next
 - (c) Travel to next port: **If** $i < L - 1$ then $next_port \leftarrow R[i + 2]$ **else** $next_port \leftarrow 0$
 - (d) **For each** state in dp_next : subtract travel cost, update $dp[i + 1]$ if feasible
5. **Return** $\max\{dp[L][c][\vec{\ell}] : \text{all feasible states}\}$

4.3.3 Optimization Strategies

1. **Item Independence:** If items don't share capacity constraints (each has separate capacity B_j), solve m independent 1D DPs and combine results. This reduces complexity from $O(B^m)$ to $O(m \times B)$.
2. **State Discretization:** For large capital ranges, discretize into bands of width Δ :

$$\text{band}(c) = \lfloor c/\Delta \rfloor$$

This reduces the number of capital states from potentially thousands to a manageable constant.

3. **Sparse Representation:** Use dictionaries (hash maps) instead of dense arrays when the state space is sparse, which is common when many states are infeasible.
4. **Early Pruning:** If a route's PCTSP bound is less than the current best solution (even with optimal transactions), skip the DP computation entirely.

4.4 Main Algorithm Integration

Function TwoPhaseHybridSolve(instance, timeout, beam_width K):

1. $start_time \leftarrow$ current time, $best_solution \leftarrow \{\text{capital} : -\infty, \text{route} : \text{None}, \text{decisions} : \text{None}\}$
2. **Phase 1:** $routes \leftarrow \text{GeneratePromisingRoutes}(\text{instance}, K)$
3. **Phase 2: For each** route R in $routes$:
 - **If** timeout exceeded: **break**
 - $(max_capital, decisions) \leftarrow \text{SolveTransactionsDP}(R, \text{instance})$
 - **If** $max_capital > best_solution.\text{capital}$: update $best_solution$
4. **Return** $best_solution$

4.5 Complexity Analysis

- **Phase 1 - Route Generation:**

- Beam search explores $O(K \times n)$ states at each depth
- Maximum depth is $O(n)$
- Total: $O(K \times n^2)$ route evaluations
- Each bound calculation: $O(n)$
- Phase 1 total: $O(K \times n^3)$
- **Phase 2 - Transaction DP:**
 - For each route of length L : $O(L \times m \times k \times B^m \times K_{capital})$
 - With K routes: $O(K \times L \times m \times k \times B^m \times K_{capital})$
 - In worst case $L = O(n)$, so: $O(K \times n \times m \times k \times B^m \times K_{capital})$
- **Overall Complexity:** $O(K \times n^3 + K \times n \times m \times k \times B^m \times K_{capital})$
- **Key Advantage:** Unlike brute force's $O(n! \times (k+1)^{m \cdot n})$, this is polynomial in K (which we control) and the fixed parameters m , k , B , and $K_{capital}$.

5 Implementation Details

5.1 Data Structures

The implementation uses the following key data structures:

- **Instance Format:** Extended dictionary format supporting multiple items:
 - `purchase_prices[port][item]`: Purchase price for item type at port
 - `sale_prices[port][item]`: Sale price for item type at port
 - Both stored as dictionaries mapping port indices to lists of prices
- **State Representation:**
 - Capital discretization: Capital values are grouped into bands of width $\Delta = \max(1.0, \text{initial_capital}/100)$
 - Load vectors: Tuples of integers $(l_1, l_2, \dots, l_m)$ representing cargo for each item type
 - Sparse DP tables: Dictionaries mapping $(\text{capital_band}, \text{load_tuple})$ to $(\text{best_capital}, \text{prev_state}, \dots)$
- **Route Representation:** Lists of port indices $[v_0, v_1, \dots, v_L, v_0]$ with start and end at Amsterdam

5.2 Key Optimizations

5.2.1 Capital Discretization

To manage the potentially large capital state space, we discretize capital into bands:

$$\text{band}(c) = \lfloor c/\Delta \rfloor$$

where $\Delta = \max(1.0, r/100)$ adapts to the initial capital r . This reduces the number of capital states from potentially thousands to approximately 100-200 bands, while maintaining sufficient precision for optimization.

5.2.2 Sparse State Representation

Instead of dense arrays, we use dictionaries (hash maps) to store DP states. This is efficient because:

- Many states are infeasible and never reached
- State space is naturally sparse, especially with capacity constraints
- Memory usage scales with reachable states, not theoretical maximum

5.2.3 Beam Search Pruning

The beam search in Phase 1 uses several pruning strategies:

1. **Time-based pruning:** Routes exceeding T_{max} are discarded immediately
2. **Bound-based pruning:** Routes with PCTSP bound below threshold are skipped
3. **Beam width limit:** Only top K candidates are kept at each depth

5.2.4 Early Termination

In Phase 2, if a route's PCTSP bound is worse than the current best solution (even with optimal transactions), the DP computation is skipped entirely.

5.3 Complexity Analysis

5.3.1 Time Complexity

- **Phase 1 - Route Generation:** $O(K \times n^3)$
 - K : Beam width (controlled parameter)
 - n : Number of ports
 - Bound calculation: $O(n)$ per route
 - Total routes considered: $O(K \times n)$
- **Phase 2 - Transaction DP:** $O(K \times L \times m \times k \times B^m \times K_{capital})$
 - K : Number of routes evaluated
 - L : Route length (typically $O(n)$)
 - m : Number of item types
 - k : Maximum units per operation
 - B : Capacity
 - $K_{capital}$: Number of capital bands (approximately 100-200)
- **Overall:** $O(K \times n^3 + K \times n \times m \times k \times B^m \times K_{capital})$
- **Key Advantage:** Unlike brute force's $O(n! \times (k + 1)^{m \cdot n})$, this is polynomial in the controlled parameter K and fixed parameters $m, k, B, K_{capital}$.

5.3.2 Space Complexity

- **Route Generation:** $O(K \times n)$ for beam storage
- **Transaction DP:** $O(L \times K_{capital} \times (B + 1)^m)$ for DP tables
- **Overall:** $O(K \times n + L \times K_{capital} \times (B + 1)^m)$

6 Experimental Results

6.1 Test Instances

We tested the algorithm on instances with:

- Port counts: $n \in \{15, 20, 25, 30\}$
- Item types: $m = 2$ items per port
- Operations: $k = 2$ units per buy/sell operation
- Capacity: $B = \min(5, n - 1)$
- Time limit: $T_{max} = 20 + (n - 2) \times 10$
- Initial capital: $r = 100 + (n - 3) \times 20$

6.2 Performance Metrics

Table 1 presents execution results for various instance sizes.

Table 1: Algorithm Performance on Test Instances

n	Routes Generated	Routes Evaluated	Execution Time (s)	Final Capital	Timeout
15	1,141	1	0.43	593.0	No
20	2,569	1	0.96	847.2	No
25	4,424	1	1.78	1,071.6	No
30	5,429	52	61.34	1,297.2	No

6.3 Observations

1. **Scalability:** The algorithm successfully handles instances up to $n = 30$ within reasonable time (under 62 seconds for $n = 30$).
2. **Route Generation Efficiency:** Beam search effectively limits route exploration. For $n = 30$, only 5,429 routes were generated (compared to $30! \approx 2.65 \times 10^{32}$ possible routes).
3. **DP Efficiency:** Most routes are pruned by PCTSP bounds, so only a small fraction require DP evaluation. For $n \leq 25$, only 1 route needed DP evaluation, indicating effective pruning.
4. **Execution Time Growth:** Time grows sub-exponentially with n :
 - $n = 15 \rightarrow 20$: $2.2\times$ increase
 - $n = 20 \rightarrow 25$: $1.9\times$ increase

- $n = 25 \rightarrow 30$: $34.5\times$ increase (due to more DP evaluations)

5. **Solution Quality:** The algorithm finds profitable solutions for all test instances, with capital increasing with instance size (as expected with more ports and higher initial capital).

6.4 Comparison with Theoretical Bounds

The PCTSP bounds provide effective pruning:

- For $n \leq 25$, the first route generated had a bound high enough that no other routes needed evaluation
- For $n = 30$, 52 routes were evaluated, indicating the bound becomes less selective for larger instances
- The bound calculation overhead is minimal ($O(n)$ per route), making it an efficient pruning tool

6.5 Correctness Validation

We validated correctness on small instances ($n \in \{3, 4, 5\}$) where solutions can be verified:

- All solutions are feasible (satisfy capacity, capital, and time constraints)
- Routes are valid (start and end at Amsterdam, no repeated ports)
- Solutions are profitable (final capital $>$ initial capital for feasible instances)

7 Conclusions

7.1 Algorithm Effectiveness

The two-phase hybrid algorithm successfully addresses the challenge of solving the Dutch Merchant Problem for medium-to-large instances ($15 \leq n \leq 30$) with multiple items and k -unit operations. Key achievements:

1. **Scalability:** Handles instances up to $n = 30$ within reasonable time (under 62 seconds)

2. **Efficiency:** Reduces route exploration from $O(n!)$ to $O(K \times n)$ where K is controlled
3. **Optimality:** For each generated route, finds optimal transaction decisions using exact DP
4. **Flexibility:** Supports variable numbers of items (m) and operation sizes (k)

7.2 Scalability Analysis

The algorithm's performance demonstrates:

- **Sub-exponential growth:** Execution time grows much slower than brute force's $O(n!)$
- **Controlled exploration:** Beam width K provides a tunable parameter balancing solution quality vs. computation time
- **Effective pruning:** PCTSP bounds eliminate most routes without DP evaluation
- **Practical limits:** For $n > 30$, increasing beam width or using more sophisticated route generation heuristics may be necessary

7.3 Theoretical Contributions

This work demonstrates:

1. **Separation of Concerns:** Separating route generation (NP-hard) from transaction optimization (polynomial with DP) is an effective strategy
2. **PCTSP Connection:** Leveraging the problem's relationship to Prize-Collecting TSP provides natural bounds and heuristics
3. **DP Applicability:** The transaction subproblem exhibits optimal substructure, making DP the appropriate technique
4. **Hybrid Approach:** Combining heuristic route generation with exact transaction optimization balances efficiency and solution quality

7.4 Future Improvements

Potential enhancements for even larger instances:

1. **Adaptive Beam Width:** Dynamically adjust beam width based on bound distribution
2. **Item Independence Exploitation:** Detect and exploit cases where items have separate capacity constraints
3. **Parallel Route Evaluation:** Evaluate multiple routes' DP problems in parallel
4. **Advanced Route Heuristics:** Use more sophisticated PCTSP approximation algorithms for route generation
5. **State Space Reduction:** Further optimize DP state representation for very large m or B

7.5 Connection to Project Requirements

This implementation fulfills the requirements for an efficient algorithm that:

- Handles instances with $15 \leq n \leq 30$ ports
- Supports multiple items per port (m items)
- Allows k -unit buy/sell operations
- Uses dynamic programming for transaction optimization
- Leverages PCTSP-inspired heuristics for route generation
- Provides theoretical justification through complexity analysis and connection to established problems

The algorithm serves as a practical solution for medium-to-large instances where exact brute force methods become intractable, while maintaining theoretical foundations in both Prize-Collecting TSP and Dynamic Programming literature.

References

- [1] Alberto Marchetti-Spaccamela, Vincenzo Bonifaci, Stefano Leonardi, and Giorgio Ausiello. Prize-collecting traveling salesman and related problems. In *Handbook of Approximation Algorithms and Metaheuristics*, 2007.